

LET'S
ROCK

THE
FUTURE

FIAP

GRADUAÇÃO

CURSOS PRESENCIAIS E ON-LINE

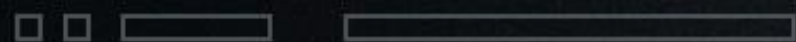
Prof. Dr. Marcus Grilo

Prof. Dr. Marcus Grilo

Ciência da Computação

Computer Organization and Architecture

Processadores e Ciclo de Execução



Prof. Dr. Marcus Grilo



<https://www.linkedin.com/in/marcus-grilo/>



profmarcus.grilo@fiap.com.br

Ao final da aula, você deverá ser capaz de:

- Identificar os principais componentes de uma CPU.
- Entender o ciclo **Fetch → Decode → Execute**.
- Relacionar clock e execução de instruções.
- Compreender a função dos registradores.
- Perceber a diferença entre uma instrução de alto nível em Python e as operações realizadas pelo processador.
- Utilizar o Raspberry Pi Pico para criar uma **simulação didática do ciclo de execução**.

O que o processador realmente faz?

Quando o Python executa $c = a + b$, o que realmente acontece dentro do computador?

```
</> Python
```

```
a = 10
```

```
b = 5
```

```
c = a + b
```

Programa



Instruções



CPU



Unidade de Controle

Registradores

ULA

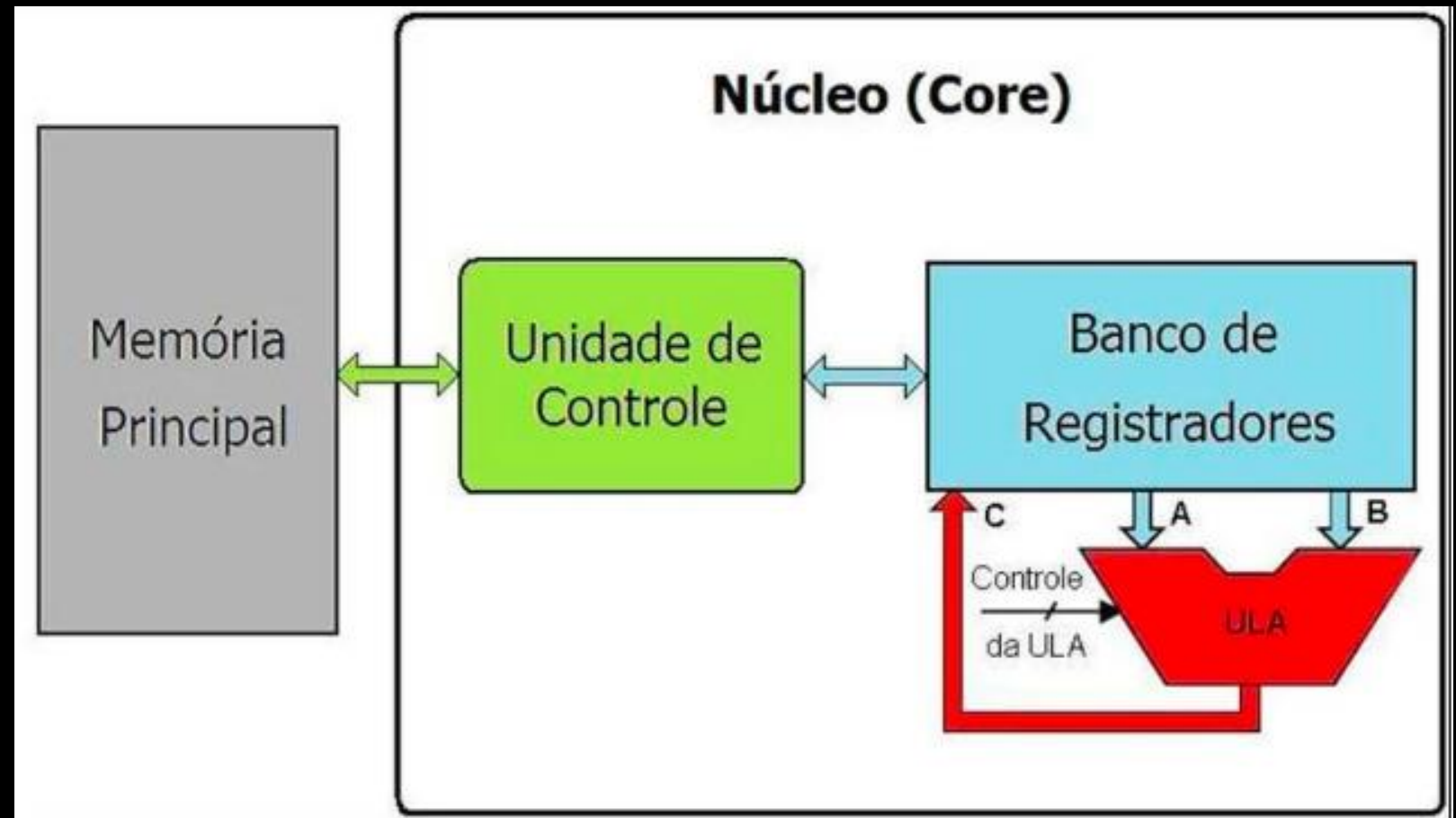
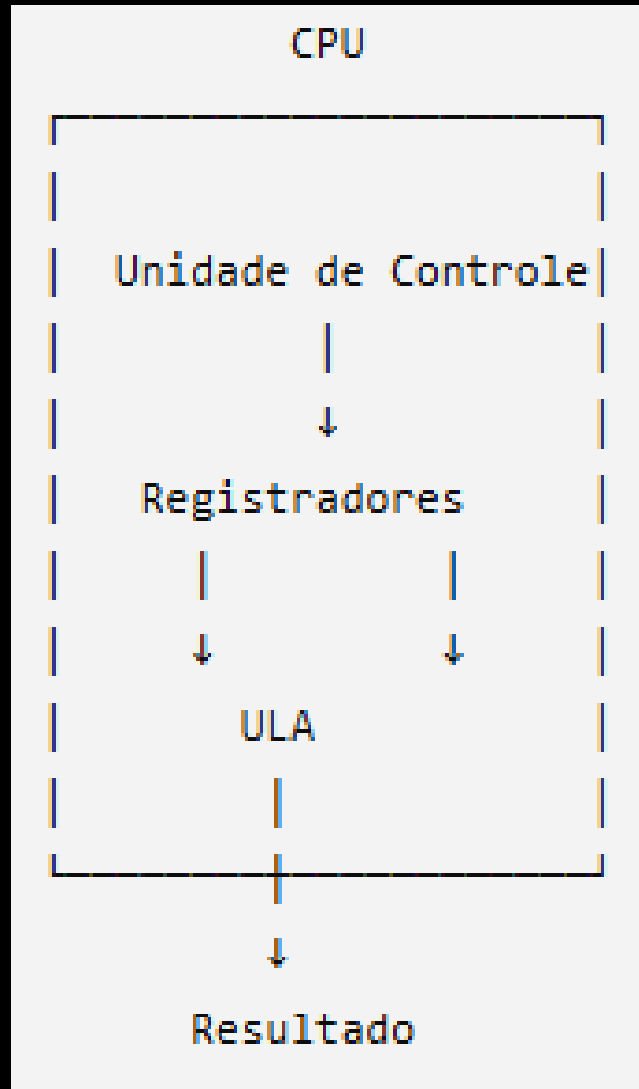
Clock



Resultado

Arquitetura simplificada da CPU

Componentes necessários



Arquitetura simplificada da CPU

Unidade de Controle

- Coordena a execução.
- Busca e interpreta instruções.

Registradores

- Pequenas áreas de armazenamento dentro da CPU.
- Guardam valores temporários e informações necessárias à execução.

ULA

- Realiza operações aritméticas e lógicas.

Clock

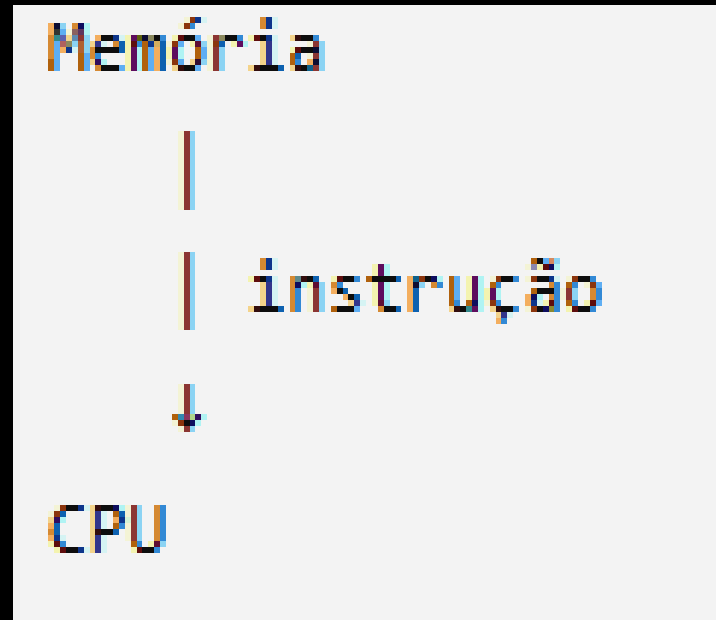
- Fornece uma referência temporal para sincronizar operações.

O ciclo de execução

FETCH → DECODE → EXECUTE

1. FETCH — Busca

A CPU busca a próxima instrução na memória.



O ciclo de execução

FETCH → DECODE → EXECUTE

2. DECODE — Decodificação

A unidade de controle interpreta a instrução.

Exemplo:

```
ADD R1, R2
```

Significa, de forma simplificada:

"Some o conteúdo de R1 com R2."

O ciclo de execução

FETCH → DECODE → EXECUTE

3. EXECUTE — Execução

A ULA realiza a operação.

R1 + R2

↓

ULA

↓

Resultado

O ciclo de execução

FETCH → DECODE → EXECUTE

FETCH



DECODE



EXECUTE



FETCH



DECODE



EXECUTE

...

O ciclo de execução

Vamos utilizar uma sequência de "instruções".

```
LOAD  
ADD  
STORE
```

Por exemplo:

```
LOAD R0, 5  
LOAD R1, 3  
ADD R0, R1  
STORE R2
```



```
R0 = 5  
R1 = 3  
R2 = 8
```

Simulando FETCH → DECODE → EXECUTE

Programa

```
import time

# Memória de instruções
programa = [
    "LOAD R0 5",
    "LOAD R1 3",
    "ADD R0 R1",
    "STORE R0 R2"
]

# Registradores
registradores = {
    "R0": 0,
    "R1": 0,
    "R2": 0
}

# Program Counter
pc = 0

while pc < len(programa):
    print()
    print("=====")

    # =====
    # FETCH
    # =====
    instrucao = programa[pc]

    print("FETCH")
    print("PC =", pc)
    print("Instrucao:", instrucao)

    time.sleep(1)

    # =====
    # DECODE
    # =====
    partes = instrucao.split()

    comando = partes[0]

    print()
    print("DECODE")

    print("Comando:", comando)

    time.sleep(1)

    # =====
    # EXECUTE
    # =====

    print()
    print("EXECUTE")

    if comando == "LOAD":

        registrador = partes[1]
        valor = int(partes[2])

        registradores[registrador] = valor

        print("Carregando", valor, "em", registrador)

    elif comando == "ADD":

        r1 = partes[1]
        r2 = partes[2]

        registradores[r1] = registradores[r1] + registradores[r2]

        print(r1, "+", r2)

    elif comando == "STORE":

        origem = partes[1]
        destino = partes[2]

        registradores[destino] = registradores[origem]

        print("Copiando", origem, "para", destino)

    print()
    print("REGISTRADORES")
    print("R0 =", registradores["R0"])
    print("R1 =", registradores["R1"])
    print("R2 =", registradores["R2"])

    print("=====")

    time.sleep(1)

    # Próxima instrução
    pc += 1
```

Simulando FETCH → DECODE → EXECUTE FIAP

Programa

=====	EXECUTE	=====
FETCH	Carregando 3 em R1	=====
PC = 0		=====
Instrucao: LOAD R0 5		
	REGISTRADORES	FETCH
DECODE	R0 = 5	PC = 3
Comando: LOAD	R1 = 3	Instrucao: STORE R0 R2
	R2 = 0	
	=====	
EXECUTE		DECODE
Carregando 5 em R0		Comando: STORE
REGISTRADORES	FETCH	EXECUTE
R0 = 5	PC = 2	Copiando R0 para R2
R1 = 0	Instrucao: ADD R0 R1	
R2 = 0		
=====	DECODE	REGISTRADORES
	Comando: ADD	R0 = 8
=====		R1 = 3
FETCH	EXECUTE	R2 = 8
PC = 1	R0 + R1	=====
Instrucao: LOAD R1 3		
	REGISTRADORES	
DECODE	R0 = 8	
Comando: LOAD	R1 = 3	
	R2 = 0	
EXECUTE	=====	

Simulando **FETCH** → **DECODE** → **EXECUTE**^{FIA/P}

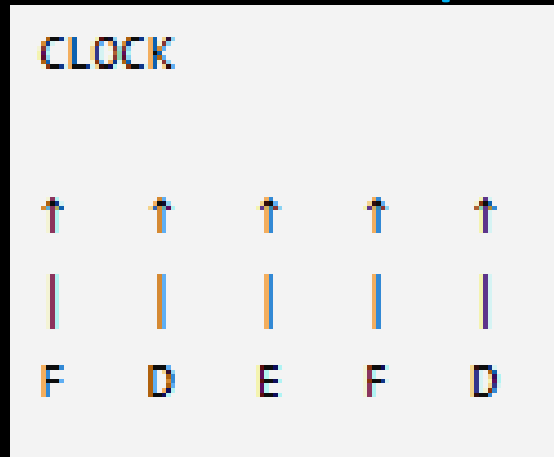
Programa permite fazer a associação:

MicroPython	CPU
programa	Memória
pc	Program Counter
instrucao	Instrução
registradores	Registradores
split()	Decodificação
+	Operação da ULA
while	Repetição do ciclo

Simulando FETCH → DECODE → EXECUTE_{FIAP}

Por que o processador precisa de um clock?

O clock fornece uma referência para sincronizar as operações internas.



No MicroPython podemos fazer uma analogia simples:

```
</> Python
```

```
time.sleep(1)
```

⚠ `time.sleep(1)` não é o clock real do processador. É apenas uma forma de tornar o processo visível na simulação.

Simulando FETCH → DECODE → EXECUTE FIAP

O papel do Program Counter

O Program Counter (PC) indica qual instrução deve ser buscada em seguida.

Memória

```
0 → LOAD R0,5
1 → LOAD R1,3
2 → ADD R0,R1
3 → STORE R2
```

Quando:

```
</> Python
```

```
pc = 0
```

a CPU busca:

```
LOAD R0,5
```

Depois:

```
</> Python
```

```
pc = 1
```

Busca:

```
LOAD R1,3
```

E assim por diante.

Resumo

Agora que construímos uma CPU simplificada, o que acontece quando executamos essa operação?

```
</> Python
```

```
a = 10
```

```
b = 5
```

```
c = a + b
```

Um processador não entende diretamente o nosso código Python. Ele executa instruções, uma após outra, coordenadas por um ciclo de execução.

Código Python



Interpretador / ambiente de execução



Instruções



CPU



FETCH



DECODE



EXECUTE



Registradores / ULA / memória



Resultado

Agora é com você

1 - Objetivo: modificar o simulador para executar novas instruções.

Deverá ser implementado: SUB e MUL e acrescentar LEDs para mostrar qual etapa está acontecendo.

LED 1 → FETCH	FETCH
LED 2 → DECODE	
LED 3 → EXECUTE	DECODE
	
	EXECUTE
	

Por exemplo:

```
LOAD R0, 10  
LOAD R1, 2  
SUB R0, R1
```

R0 = 8

```
LOAD R0, 4  
LOAD R1, 3  
MUL R0, R1
```

R0 = 12